1470. Shuffle the Array

Easy | Topics | Companies | Hint

Given the array `nums` consisting of `2n` elements in the form `[x1,x2,...,xn,y1,y2,...,yn]`.

Return the array in the form `[x1,y1,x2,y2,...,xn,yn]`.

Example 1:

```
Input: nums = [2,5,1,3,4,7], n = 3
Output: [2,3,5,4,1,7]
Explanation: Since x1=2, x2=5, x3=1, y1=3, y2=4, y3=7 then the answer is [2,3,5,4,1,7].
```

Example 2:

```
Input: nums = [1,2,3,4,4,3,2,1], n = 4
Output: [1,4,2,3,3,2,4,1]
```

Example 3:

```
Input: nums = [1,1,2,2], n = 2
Output: [1,2,1,2]
```

Constraints:

- `1 <= n <= 500`
- `nums.length == 2n`
- `1 <= nums[i] <= 10^3`

Discover more | Computer Education

534 | 42 | 166

```java
class Solution {
    public int[] shuffle(int[] nums, int n) {
        int[] ans = new int[2 * n];
        int index = 0;

        for (int i = 0; i < n; i++) {
            ans[index++] = nums[i];
            ans[index++] = nums[i + n];
        }

        return ans;
    }
}
```

Testcase | Test Result

Accepted

Case 1 | Case 2 | Case 3

Input

nums =
[2,5,1,3,4,7]

n =
3

Output

[2,3,5,4,1,7]

Expected

[2,3,5,4,1,7]

Problem List

Description | Editorial | Solutions | Submissions

1480. Running Sum of 1d Array

Easy | Topics | Companies | Hint

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]…nums[i])`.

Return the running sum of `nums`.

Example 1:

```
Input: nums = [1,2,3,4]
Output: [1,3,6,10]
Explanation: Running sum is obtained as follows: [1, 1+2, 1+2+3, 1+2+3+4].
```

Example 2:

```
Input: nums = [1,1,1,1,1]
Output: [1,2,3,4,5]
Explanation: Running sum is obtained as follows: [1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1].
```

Example 3:

```
Input: nums = [3,1,2,10,1]
Output: [3,4,6,16,17]
```

Constraints:

- `1 <= nums.length <= 1000`
- `-10^6 <= nums[i] <= 10^6`

8.9K | 221 | 142 Online

Code

Java | Auto

```java
class Solution {
    public int[] runningSum(int[] nums) {
        for (int i = 1; i < nums.length; i++) {
            nums[i] = nums[i] + nums[i - 1];
        }
        return nums;
    }
}
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =
[1,2,3,4]

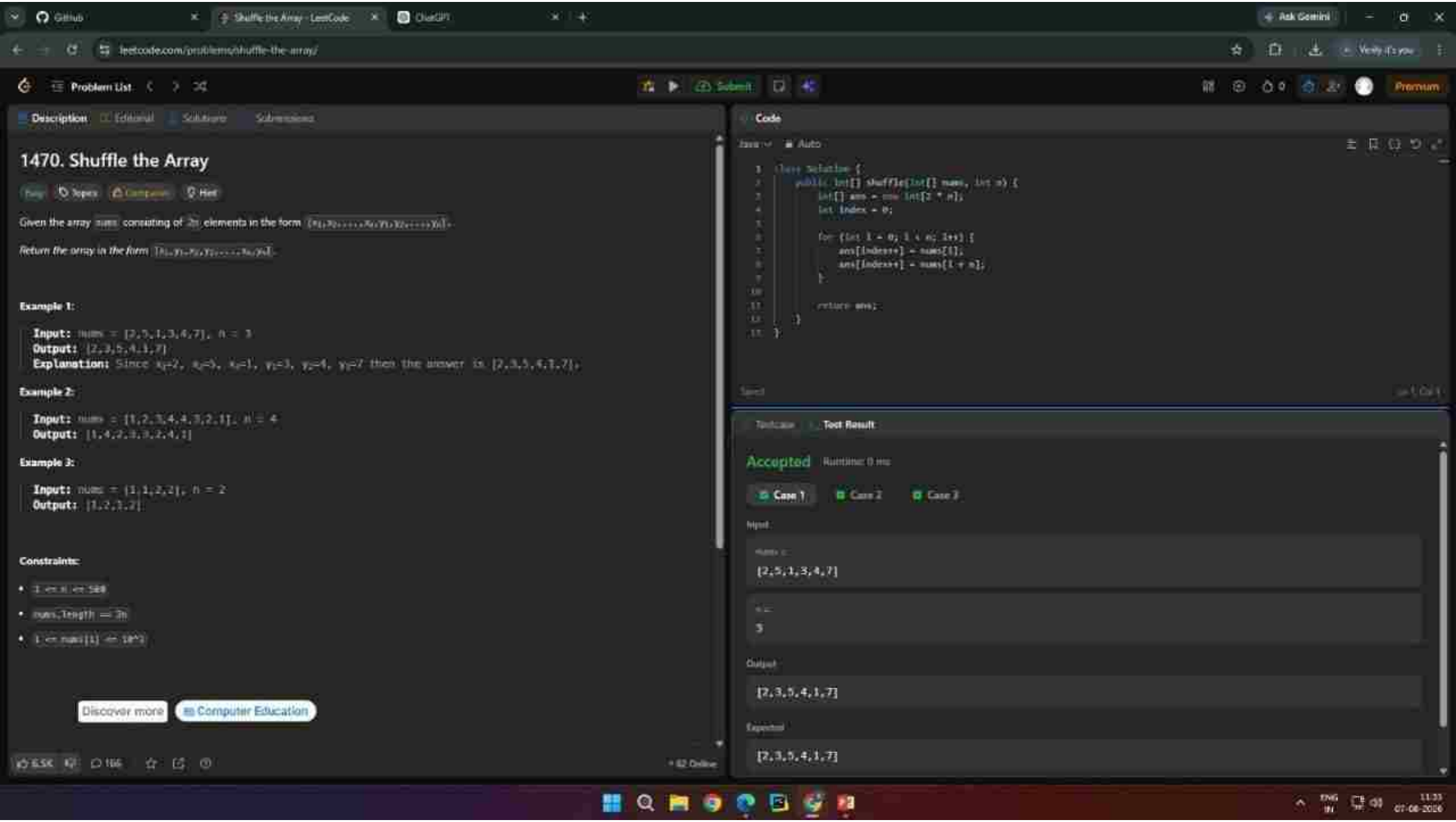
Output

[1,3,6,10]

Expected

[1,3,6,10]

Contribute a testcase


Github
Shuffle the Array - LeetCode
ChatGPT
Ask Gemini
leetcode.com/problems/shuffle-the-array/
Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
1470. Shuffle the Array
Topics
Companies
Hint
Given the array nums consisting of 2n elements in the form [x1,x2,...,xn,y1,y2,...,yn].
Return the array in the form [x1,y1,x2,y2,...,xn,yn].
Example 1:
Input: nums = [2,5,1,3,4,7], n = 3
Output: [2,3,5,4,1,7]
Explanation: Since x1=2, x2=5, x3=1, y1=3, y2=4, y3=7 then the answer is [2,3,5,4,1,7].
Example 2:
Input: nums = [1,2,3,4,4,3,2,1], n = 4
Output: [1,4,2,3,3,2,4,1]
Example 3:
Input: nums = [1,1,2,2], n = 2
Output: [1,2,1,2]
Constraints:
1 <= n <= 500
nums.length == 2n
1 <= nums[i] <= 10^3
Discover more
Computer Education
6.5K
166
82 Online
Code
Java
Auto
class Solution {
public int[] shuffle(int[] nums, int n) {
int[] ans = new int[2 * n];
int index = 0;
for (int i = 0; i < n; i++) {
ans[index++] = nums[i];
ans[index++] = nums[i + n];
}
return ans;
}
}
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[2,5,1,3,4,7]
n =
3
Output
[2,3,5,4,1,7]
Expected
[2,3,5,4,1,7]

Problem List

Description | Editorial | Solutions | Submissions

1732. Find the Highest Altitude

Easy | Topics | Companies | Hint

There is a biker going on a road trip. The road trip consists of `n + 1` points at various altitudes. The biker starts his trip on point `0` with altitude equal `0`.

You are given an integer array `gain` of length `n` where `gain[i]` is the **net gain in altitude** between points `i` and `i + 1` for all `(0 <= i < n)`. Return the ***highest altitude*** *of a point.*

Example 1:

```
Input: gain = [-5,1,5,0,-7]
Output: 1
Explanation: The altitudes are [0,-5,-4,1,1,-6]. The highest is 1.
```

Example 2:

```
Input: gain = [-4,-3,-2,-1,4,3,2]
Output: 0
Explanation: The altitudes are [0,-4,-7,-9,-10,-6,-3,-1]. The highest is 0.
```

Constraints:

- `n == gain.length`
- `1 <= n <= 100`
- `-100 <= gain[i] <= 100`

Discover more | Search Engines

Seen this question in a real interview before? 1/6

3.5K | 427 | + 50 Online

Code

Java | Auto

```java
class Solution {
    public int largestAltitude(int[] gain) {
        int altitude = 0;
        int maxAltitude = 0;

        for (int g : gain) {
            altitude += g;
            maxAltitude = Math.max(maxAltitude, altitude);
        }

        return maxAltitude;
    }
}
```

Saved

Testcase | Test Result

Accepted | Runtime: 0 ms

Case 1 | Case 2

Input

gain =
[-5,1,5,0,-7]

Output

1

Expected

1

Contribute a testcase

Description | Editorial | Solutions | Submissions

49. Group Anagrams

Medium | Topics | Companies

Given an array of strings `strs`, group the anagrams together. You can return the answer in **any order**.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in strs that can be rearranged to form `"bat"`.
- The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.
- The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.

Example 2:

Input: `strs = [""]`

Output: `[[""]]`

Example 3:

Input: `strs = ["a"]`

Output: `[["a"]]`

Constraints:

- `1 <= strs.length <= 10^4`
- `0 <= strs[i].length <= 100`
- `strs[i]` consists of lowercase English letters.

22.4K | 425 | 422 Online

Code

Java | Auto

```java
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();

        for (String str : strs) {
            char[] chars = str.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);

            map.putIfAbsent(key, new ArrayList<>());
            map.get(key).add(str);
        }

        return new ArrayList<>(map.values());
    }
}
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

strs =
["eat","tea","tan","ate","nat","bat"]

Output

[["eat","tea","ate"],["bat"],["tan","nat"]]

Expected

[["bat"],["nat","tan"],["ate","eat","tea"]]

Contribute a testcase

Problem List

Description | Editorial | Solutions | Submissions

724. Find Pivot Index

Easy | Topics | Companies | Hint

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is `0` because there are no elements to the left. This also applies to the right edge of the array.

Return *the **leftmost pivot index***. If no such index exists, return `-1`.

Example 1:

```
Input: nums = [1,7,3,6,5,6]
Output: 3
Explanation:
The pivot index is 3.
Left sum = nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11
Right sum = nums[4] + nums[5] = 5 + 6 = 11
```

Example 2:

```
Input: nums = [1,2,3]
Output: -1
Explanation:
There is no index that satisfies the conditions in the problem statement.
```

Example 3:

```
Input: nums = [2,1,-1]
Output: 0
Explanation:
The pivot index is 0.
Left sum = 0 (no elements to the left of index 0)
Right sum = nums[1] + nums[2] = 1 + -1 = 0
```

8.5K | 277 | 119 Online

Code

Java | Auto

```java
        int totalSum = 0;

        // Calculate total sum
        for (int num : nums) {
            totalSum += num;
        }

        int leftSum = 0;

        // Find pivot index
        for (int i = 0; i < nums.length; i++) {
            int rightSum = totalSum - leftSum - nums[i];

            if (leftSum == rightSum) {
                return i;
            }

            leftSum += nums[i];
        }

        return -1;
    }
```

Saved

Testcase | Test Result

Input

nums =

[1,7,3,6,5,6]

Output

3

Expected

3

Contribute a testcase

Problem List

Description | Editorial | Solutions | Submissions

53. Maximum Subarray

Medium | Topics | Companies

Given an integer array `nums`, find the subarray with the largest sum, and return *its sum*.

Example 1:

```
Input: nums = [-2,1,-3,4,-1,2,1,-5,4]
Output: 6
Explanation: the subarray [4,-1,2,1] has the largest sum 6.
```

Example 2:

```
Input: nums = [1]
Output: 1
Explanation: the subarray [1] has the largest sum 1.
```

Example 3:

```
Input: nums = [5,4,-1,7,8]
Output: 23
Explanation: the subarray [5,4,-1,7,8] has the largest sum 23.
```

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`

Follow up: If you have figured out the `O(n)` solution, try coding another solution using the **divide and conquer** approach, which is more subtle.

Discover more | 60 Data Structure Courses

38.2K | 372 | 683 Online

Code

Java | Auto

```java
class Solution {
    public int maxSubArray(int[] nums) {
        int maxSum = nums[0];
        int currentSum = nums[0];

        for (int i = 1; i < nums.length; i++) {
            currentSum = Math.max(nums[i], currentSum + nums[i]);
            maxSum = Math.max(maxSum, currentSum);
        }

        return maxSum;
    }
}
```

Saved | Ln 11, Col 2

Testcase | Test Result

Accepted | Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =
[-2,1,-3,4,-1,2,1,-5,4]

Output

6

Expected

6

Contribute a testcase

Problem List

Description | Editorial | Solutions | Submissions

1672. Richest Customer Wealth

Easy | Topics | Companies | Hint

You are given an `m x n` integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return the **wealth** that the richest customer has.

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum **wealth**.

Example 1:

```
Input: accounts = [[1,2,3],[3,2,1]]
Output: 6
Explanation:
1st customer has wealth = 1 + 2 + 3 = 6
2nd customer has wealth = 3 + 2 + 1 = 6
Both customers are considered the richest with a wealth of 6 each, so return 6.
```

Example 2:

```
Input: accounts = [[1,5],[7,3],[3,5]]
Output: 10
Explanation:
1st customer has wealth = 6
2nd customer has wealth = 10
3rd customer has wealth = 8
The 2nd customer is the richest with a wealth of 10.
```

Example 3:

```
Input: accounts = [[2,8,7],[7,1,3],[1,9,5]]
Output: 17
```

Constraints:

- `m == accounts.length`

498 | 110 | 50 Online

Code

Java | Auto

```java
class Solution {
    public int maximumWealth(int[][] accounts) {
        int maxWealth = 0;

        for (int[] customer : accounts) {
            int wealth = 0;

            for (int money : customer) {
                wealth += money;
            }

            maxWealth = Math.max(maxWealth, wealth);
        }

        return maxWealth;
    }
}
```

Saved | Ln 17, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

accounts =
[[1,2,3],[3,2,1]]

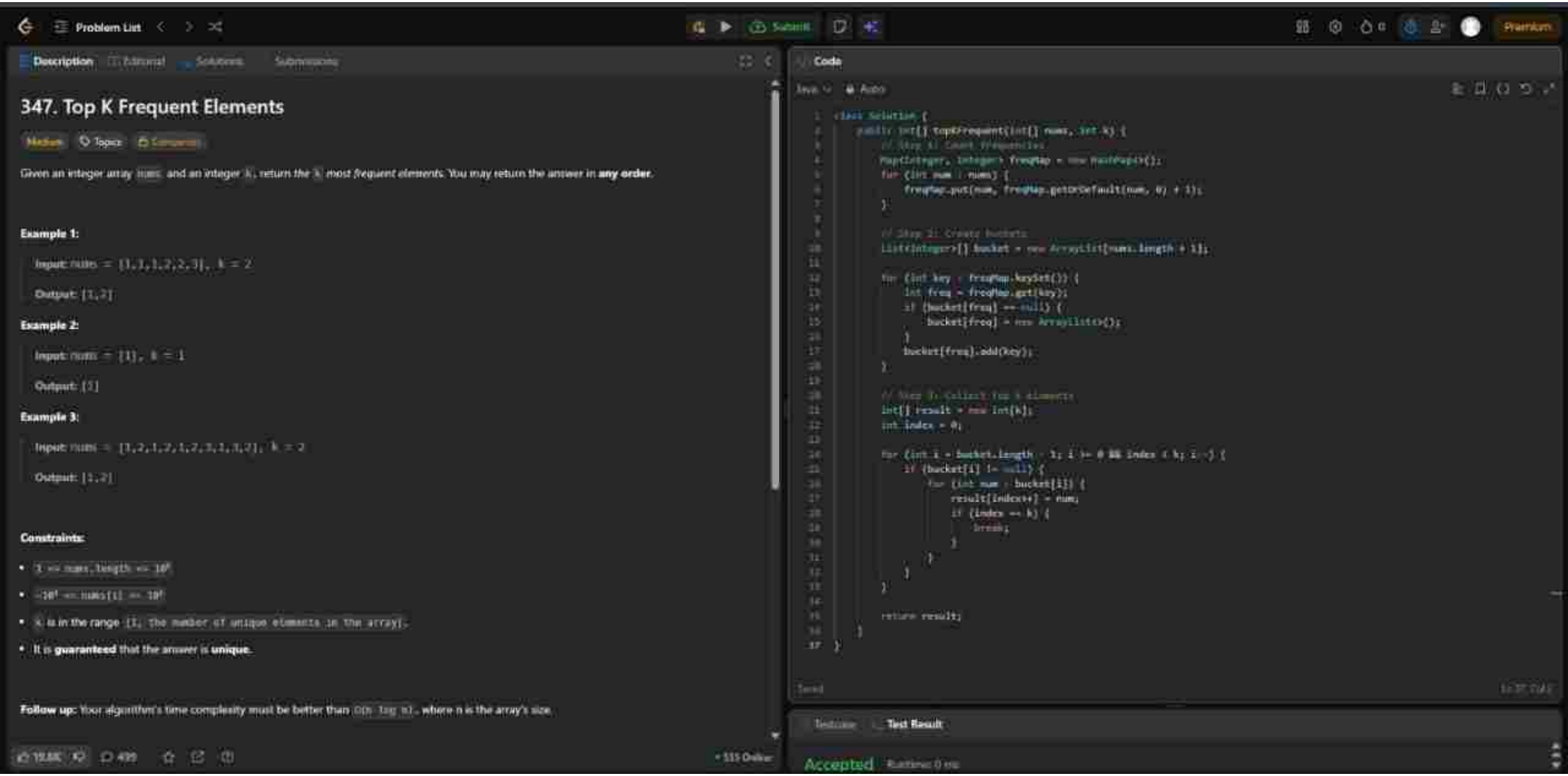
Output

6

Expected

6

Contribute a testcase


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
347. Top K Frequent Elements
Medium
Topics
Companies
Given an integer array nums and an integer k, return the k most frequent elements. You may return the answer in any order.
Example 1:
Input: nums = [1,1,1,2,2,3], k = 2
Output: [1,2]
Example 2:
Input: nums = [1], k = 1
Output: [1]
Example 3:
Input: nums = [1,2,1,2,1,2,3,1,3,2], k = 2
Output: [1,2]
Constraints:
1 <= nums.length <= 10^5
-10^4 <= nums[i] <= 10^4
k is in the range [1, the number of unique elements in the array].
It is guaranteed that the answer is unique.
Follow up: Your algorithm's time complexity must be better than O(n log n), where n is the array's size.
19.8K
499
515 Online
Code
Java
Auto
class Solution {
public int[] topKFrequent(int[] nums, int k) {
// Step 1: Count frequencies
Map<Integer, Integer> freqMap = new HashMap<>();
for (int num : nums) {
freqMap.put(num, freqMap.getOrDefault(num, 0) + 1);
}
// Step 2: Create buckets
List<Integer>[] bucket = new ArrayList[nums.length + 1];
for (int key : freqMap.keySet()) {
int freq = freqMap.get(key);
if (bucket[freq] == null) {
bucket[freq] = new ArrayList<>();
}
bucket[freq].add(key);
}
// Step 3: Collect top k elements
int[] result = new int[k];
int index = 0;
for (int i = bucket.length - 1; i >= 0 && index < k; i--) {
if (bucket[i] != null) {
for (int num : bucket[i]) {
result[index++] = num;
if (index == k) {
break;
}
}
}
}
return result;
}
}
Testcase
Test Result
Accepted
Runtime: 0 ms

Problem List

Description | Editorial | Solutions | Submissions

977. Squares of a Sorted Array

Easy | Topics | Companies

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of **the squares of each number** sorted in non-decreasing order.*

Example 1:

```
Input: nums = [-4,-1,0,3,10]
Output: [0,1,9,16,100]
Explanation: After squaring, the array becomes [16,1,0,9,100].
After sorting, it becomes [0,1,9,16,100].
```

Example 2:

```
Input: nums = [-7,-3,2,3,11]
Output: [4,9,9,49,121]
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 <= nums[i] <= 10^4`
- `nums` is sorted in **non-decreasing** order.

Follow up: Squaring each element and sorting the new array is very trivial, could you find an `O(n)` solution using a different approach?

Discover more | Computer Science

Seen this question in a real interview before? 1/6

Yes | No

10.4K | 195 | 724 Online

Code

Java | Auto

```java
class Solution {
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];

        int left = 0;
        int right = n - 1;
        int index = n - 1;

        while (left <= right) {
            int leftSquare = nums[left] * nums[left];
            int rightSquare = nums[right] * nums[right];

            if (leftSquare > rightSquare) {
                result[index] = leftSquare;
                left++;
            } else {
                result[index] = rightSquare;
                right--;
            }

            index--;
        }
    }
```

Saved | Ln 22, Col 2

Testcase | Test Result

nums =

[-4,-1,0,3,10]

Output

[0,1,9,16,100]

Expected

[0,1,9,16,100]

Contribute a testcase

Problem List

Description | Editorial | Solutions | Submissions

26. Remove Duplicates from Sorted Array

Easy | Topics | Companies | Hint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

```
Input: nums = [1,1,2]
Output: 2, nums = [1,2,_]
Explanation: Your function should return k = 2, with the first two elements of nums being 1 and 2
respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Example 2:

```
Input: nums = [0,0,1,1,1,2,2,3,3,4]
Output: 5, nums = [0,1,2,3,4,_,_,_,_,_]
```

19.5K | 1.1K | 674 Online

Code

Java | Auto

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int k = 1;

        for (int i = 1; i < nums.length; i++) {
            if (nums[i] != nums[k - 1]) {
                nums[k] = nums[i];
                k++;
            }
        }

        return k;
    }
}
```

Saved | Ln 14, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =
[1,1,2]

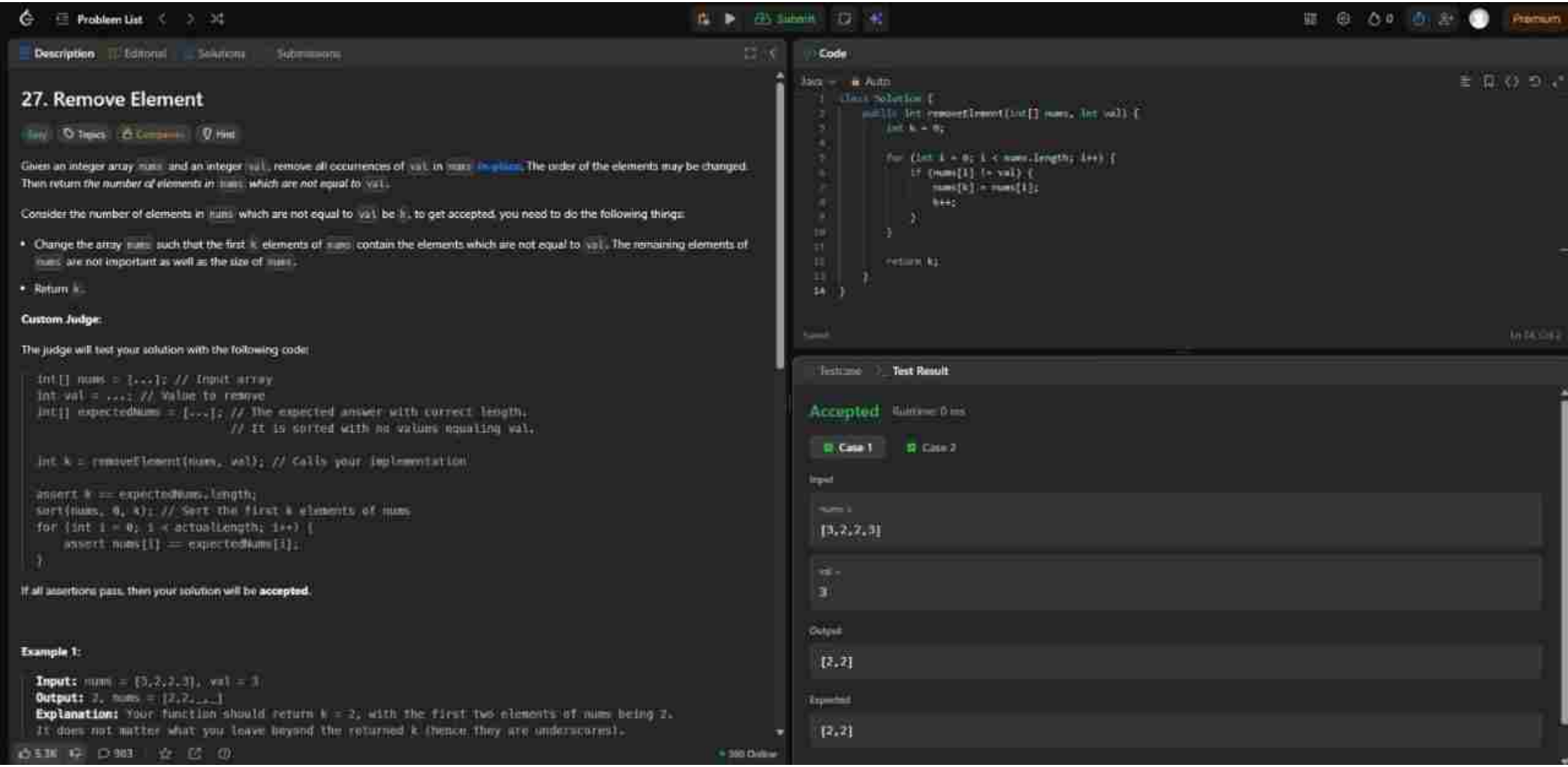
Output

[1,2]

Expected

[1,2]

Contribute a testcase


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
27. Remove Element
Topics
Hint
Given an integer array nums and an integer val, remove all occurrences of val in nums in-place. The order of the elements may be changed. Then return the number of elements in nums which are not equal to val.
Consider the number of elements in nums which are not equal to val be k, to get accepted, you need to do the following things:
Change the array nums such that the first k elements of nums contain the elements which are not equal to val. The remaining elements of nums are not important as well as the size of nums.
Return k.
Custom Judge:
The judge will test your solution with the following code:
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
// It is sorted with no values equaling val.
int k = removeElement(nums, val); // Calls your implementation
assert k == expectedNums.length;
sort(nums, 0, k); // Sort the first k elements of nums
for (int i = 0; i < actualLength; i++) {
assert nums[i] == expectedNums[i];
}
If all assertions pass, then your solution will be accepted.
Example 1:
Input: nums = [3,2,2,3], val = 3
Output: 2, nums = [2,2,_,_]
Explanation: Your function should return k = 2, with the first two elements of nums being 2.
It does not matter what you leave beyond the returned k (hence, they are underscores).
Code
Auto
class Solution {
public int removeElement(int[] nums, int val) {
int k = 0;
for (int i = 0; i < nums.length; i++) {
if (nums[i] != val) {
nums[k] = nums[i];
k++;
}
}
return k;
}
}
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[3,2,2,3]
val =
3
Output
[2,2]
Expected
[2,2]